

TRABAJO PRÁCTICO 11

Seguridad en sistemas embebidos

Incidentes y aplicaciones de criptografía

Cyberseguridad

Estudiante	Brigido Tomas Andres
Legajo	38481096
Carrera	ICOMP
Docente	Javier A. Jorge

Índice

1. Problemas de seguridad frecuentes	2
2. Incidentes en sistemas embebidos	2
2.1. Caso 1	3
2.2. Caso 2	3
2.3. Caso 3	3
3. Práctica de criptografía	4
3.1. Hash	5
3.2. Encriptación simétrica	7
3.3. Firma digital	9
Referencias	12

1 Problemas de seguridad frecuentes

1- Leer el sitio de INCIBE y revisar los problemas de seguridad más frecuentes.

Entre los problemas de seguridad revisados se destacan los siguientes:

- **Certificados autofirmados:** algunos servicios utilizan certificados sin el respaldo de una entidad de confianza. Si no se verifica su autenticidad por otro medio, puede resultar difícil confirmar la identidad del dispositivo.
- **Contraseñas embebidas y puertas traseras:** el fabricante puede incorporar contraseñas que no se pueden modificar y que se comparten entre dispositivos. Las puertas traseras permiten acceder mediante mecanismos alternativos que pueden eludir los controles configurados por el usuario.
- **Criptografía débil o inexistente:** la ausencia de cifrado o el uso de mecanismos débiles puede dejar expuesta la información almacenada o transmitida.
- **Vulnerabilidades en los servicios del dispositivo:** las interfaces HMI, de gestión y web pueden presentar fallos como XSS o inyección SQL. En estos casos, el dispositivo embebido actúa como servidor y recibe las solicitudes de los clientes.
- **Denegación de servicio (DoS y DDoS):** las solicitudes reiteradas pueden agotar los recursos del dispositivo o saturar sus comunicaciones, afectando su disponibilidad, especialmente si carece de controles adecuados.

También se identificaron otros problemas:

- Fallos en componentes de código libre.
- Autenticación débil o inexistente.
- Problemas de seguridad en las comunicaciones de entrada y salida.
- Explotación de la lógica en escalera.

2 Incidentes en sistemas embebidos

2- Buscar al menos tres casos de incidentes de seguridad en un sistema embebido e indicar cómo se pudo haber evitado.

2.1 Caso 1

Stuxnet (2010): Un gusano informático avanzado diseñado para atacar los sistemas de control industrial (ICS) y autómatas programables (PLC) de las centrifugadoras nucleares en Irán. El malware alteraba la velocidad de giro de los motores de forma imperceptible para los operadores hasta destruir el hardware.

2.2 Caso 2

Hackeo de sistemas de automóviles conectados (Jeep Cherokee, 2015): Un grupo de investigadores demostró que se podía tomar el control de un vehículo en marcha a través de su sistema de infoentretenimiento conectado a internet (Uconnect). Mediante esta brecha, lograron manipular la transmisión, los frenos y el volante.

2.3 Caso 3

Mirai (y su propagación en dispositivos IoT): Es un malware que infecta sistemas embebidos con Linux (como cámaras de seguridad IP y routers) utilizando credenciales predeterminadas de fábrica. Una vez dentro, convierte estos dispositivos en un ejército de bots (botnet) para lanzar ataques masivos de denegación de servicio (DDoS) que tiran sitios web globales. El código fuente de Mirai puede consultarse en el repositorio publicado por jgamblin [1].

3 Práctica de criptografía

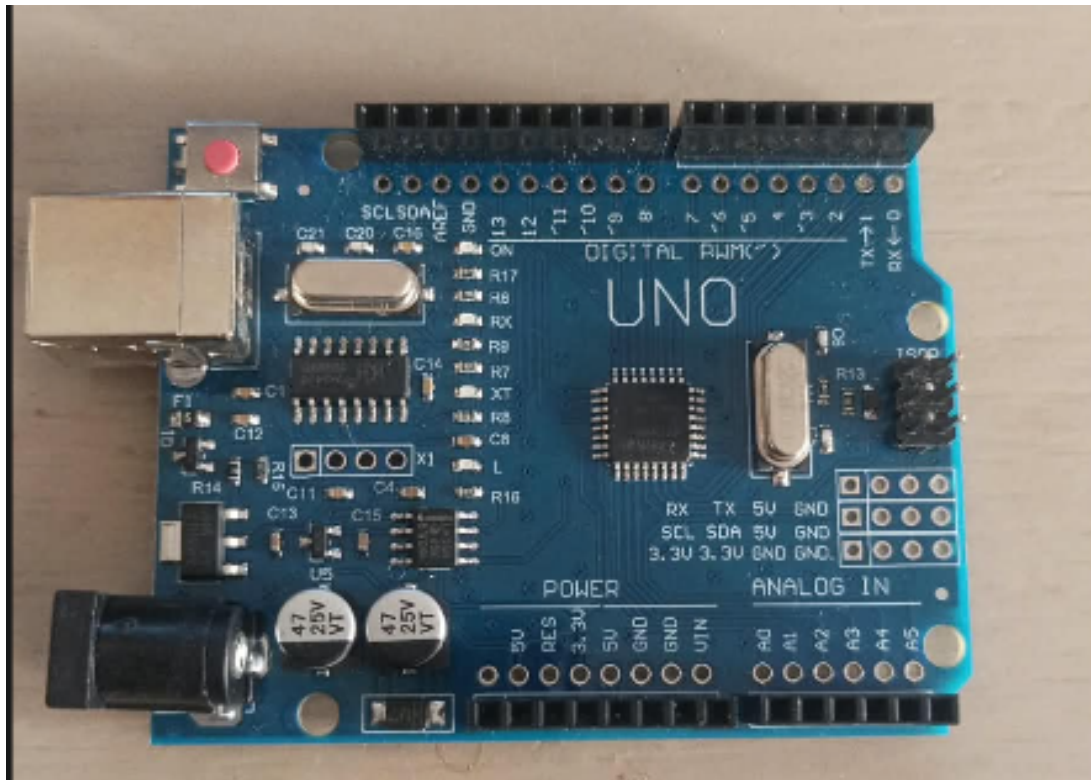
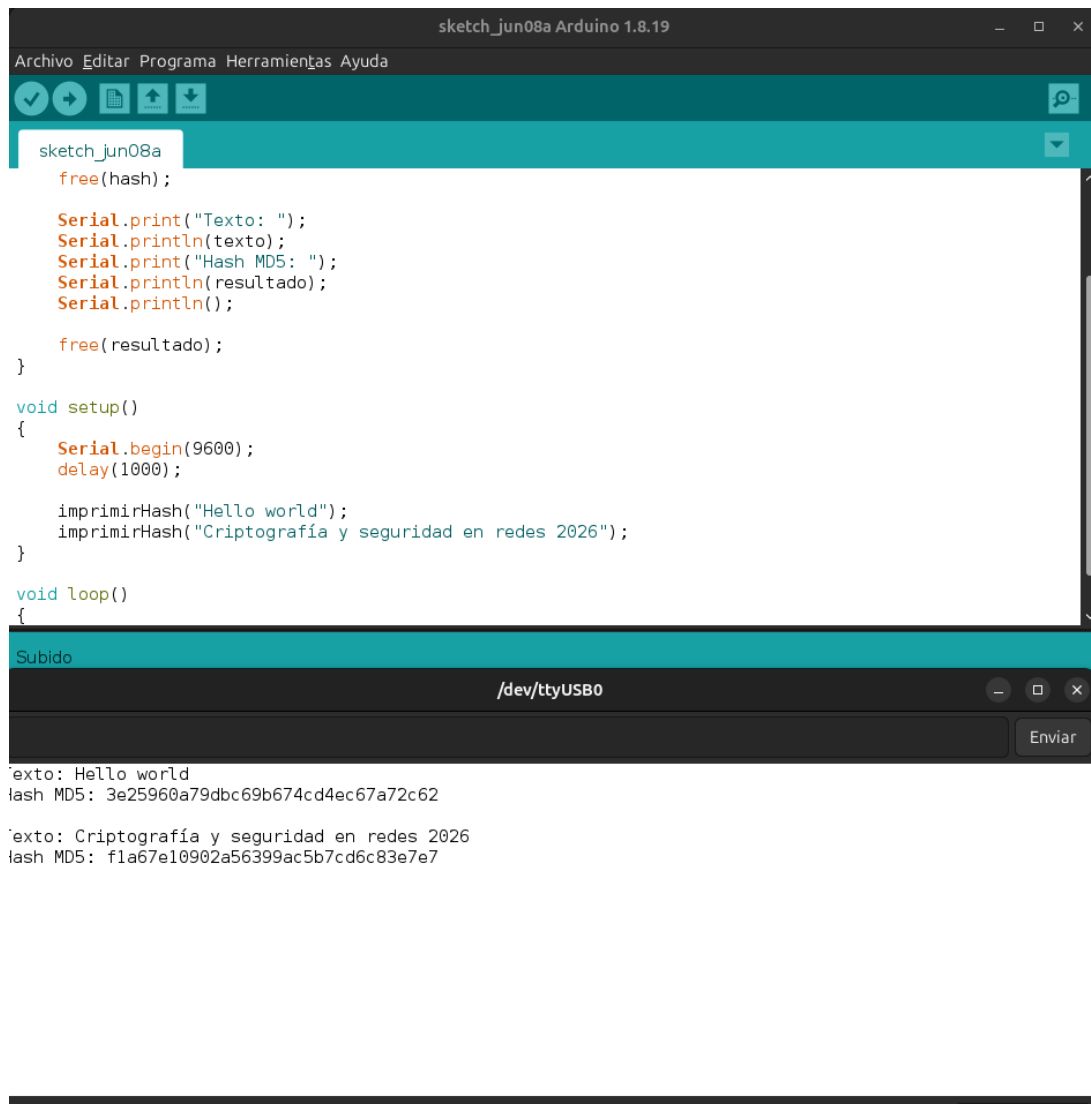


Figura 1: Placa compatible con Arduino Uno, versión SMD, utilizada para la práctica de criptografía.

3- Realizar los ejemplos de las filminas de Seguridad en sistemas embebidos: hash, encriptación simétrica y firma digital.

3.1 Hash



The image shows the Arduino IDE interface with a sketch named 'sketch_jun08a'. The code defines a function 'imprimirHash' that prints a text string and its MD5 hash. The 'setup' function initializes the serial port at 9600 baud and calls 'imprimirHash' twice with different text inputs. The 'loop' function is empty. The serial monitor shows the output of these two calls.

```
sketch_jun08a Arduino 1.8.19
Archivo Editar Programa Herramientas Ayuda

sketch_jun08a
  free(hash);

  Serial.print("Texto: ");
  Serial.println(texto);
  Serial.print("Hash MD5: ");
  Serial.println(resultado);
  Serial.println();

  free(resultado);
}

void setup()
{
  Serial.begin(9600);
  delay(1000);

  imprimirHash("Hello world");
  imprimirHash("Criptografía y seguridad en redes 2026");
}

void loop()
{
}
```

Subido

/dev/ttyUSB0

Enviar

Texto: Hello world
Hash MD5: 3e25960a79dbc69b674cd4ec67a72c62

Texto: Criptografía y seguridad en redes 2026
Hash MD5: f1a67e10902a56399ac5b7cd6c83e7e7

Figura 2: Proceso de generación de hash.

Código utilizado para generar el hash MD5.

```
#include <MD5.h>

void imprimirHash(const char* texto)
{
    unsigned char* hash = MD5::make_hash(texto);
    char* resultado = MD5::make_digest(hash, 16);
    free(hash);

    Serial.print("Texto: ");
    Serial.println(texto);
    Serial.print("Hash MD5: ");
    Serial.println(resultado);
    Serial.println();

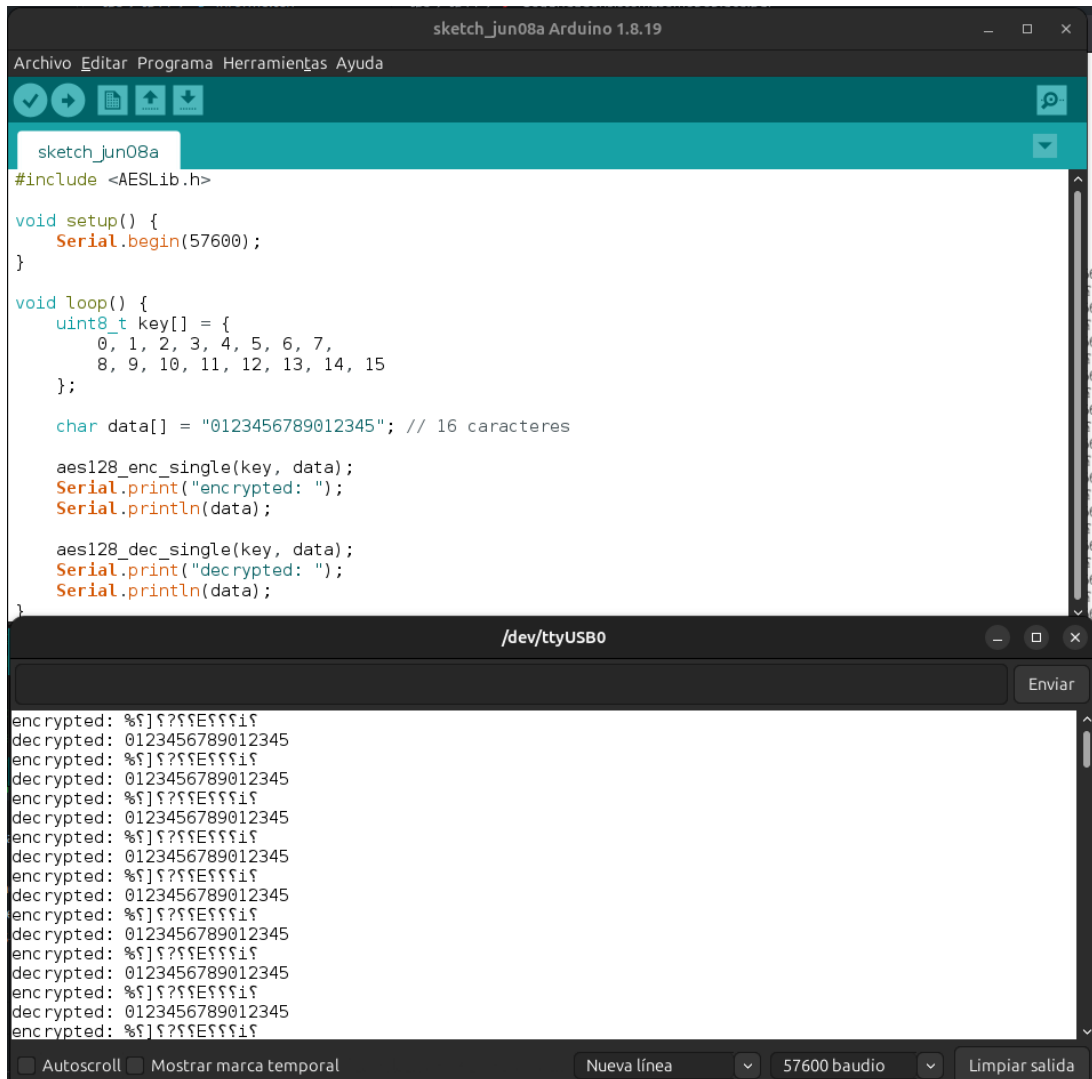
    free(resultado);
}

void setup()
{
    Serial.begin(9600);
    delay(1000);

    imprimirHash("Hello world");
    imprimirHash("Criptografía y seguridad en redes 2026");
}

void loop()
{
    // No repetir las impresiones.
}
```

3.2 Encriptación simétrica



```
sketch_jun08a Arduino 1.8.19
Archivo Editar Programa Herramientas Ayuda

sketch_jun08a
#include <AESLib.h>

void setup() {
  Serial.begin(57600);
}

void loop() {
  uint8_t key[] = {
    0, 1, 2, 3, 4, 5, 6, 7,
    8, 9, 10, 11, 12, 13, 14, 15
  };

  char data[] = "0123456789012345"; // 16 caracteres

  aes128_enc_single(key, data);
  Serial.print("encrypted: ");
  Serial.println(data);

  aes128_dec_single(key, data);
  Serial.print("decrypted: ");
  Serial.println(data);
}

/dev/ttyUSB0
Enviar

encrypted: %f]??E??i?
decrypted: 0123456789012345
encrypted: %f]??E??i?
decrypted: 0123456789012345
encrypted: %f]??E??i?
decrypted: 0123456789012345
encrypted: %f]??E??i?
decrypted: 0123456789012345
encrypted: %f]??E??i?
decrypted: 0123456789012345
encrypted: %f]??E??i?
decrypted: 0123456789012345
encrypted: %f]??E??i?
decrypted: 0123456789012345
encrypted: %f]??E??i?
decrypted: 0123456789012345
encrypted: %f]??E??i?
decrypted: 0123456789012345
encrypted: %f]??E??i?
Autoscroll Mostrar marca temporal Nueva línea 57600 baudio Limpiar salida
```

Figura 3: Proceso de encriptación simétrica.

Código utilizado para el cifrado simétrico AES. Se utiliza la biblioteca AES-Lib de DavyLandman [5].

```
#include <AESLib.h>

void setup() {
    Serial.begin(57600);
}

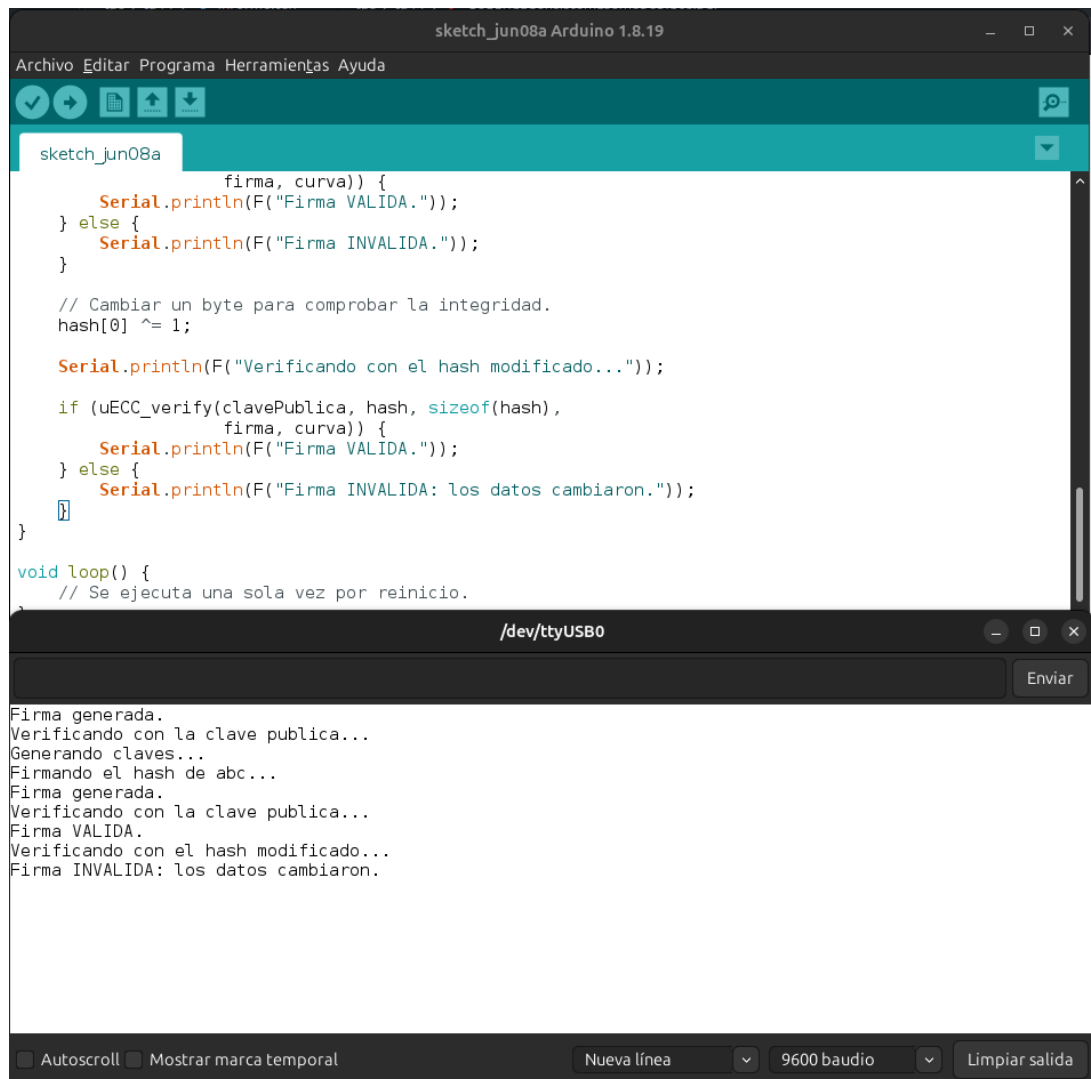
void loop() {
    uint8_t key[] = {
        0, 1, 2, 3, 4, 5, 6, 7,
        8, 9, 10, 11, 12, 13, 14, 15
    };

    char data[] = "0123456789012345"; // 16 caracteres

    aes128_enc_single(key, data);
    Serial.print("encrypted: ");
    Serial.println(data);

    aes128_dec_single(key, data);
    Serial.print("decrypted: ");
    Serial.println(data);
}
```

3.3 Firma digital



The image shows the Arduino IDE interface with a sketch named 'sketch_jun08a'. The sketch implements a digital signature verification process using the uECC library. It generates a signature for a hash, then verifies it using a public key. A second verification is performed by modifying the first byte of the hash to demonstrate that the signature is invalid for altered data.

```
sketch_jun08a
    firma, curva)) {
  Serial.println(F("Firma VALIDA."));
} else {
  Serial.println(F("Firma INVALIDA."));
}

// Cambiar un byte para comprobar la integridad.
hash[0] ^= 1;

Serial.println(F("Verificando con el hash modificado..."));

if (uECC_verify(clavePublica, hash, sizeof(hash),
  firma, curva)) {
  Serial.println(F("Firma VALIDA."));
} else {
  Serial.println(F("Firma INVALIDA: los datos cambiaron."));
}
}

void loop() {
  // Se ejecuta una sola vez por reinicio.
}
```

The serial monitor shows the following output:

```
/dev/ttyUSB0
Firma generada.
Verificando con la clave publica...
Generando claves...
Firmando el hash de abc...
Firma generada.
Verificando con la clave publica...
Firma VALIDA.
Verificando con el hash modificado...
Firma INVALIDA: los datos cambiaron.
```

Figura 4: Proceso de firma digital.

Código utilizado para la firma digital. Se utiliza la biblioteca micro-ecc de kmackay [6]. El ejemplo firma el hash SHA-256 precalculado de `abc`, verifica la firma y repite la verificación tras modificar el hash. La aleatoriedad es predecible y se utiliza únicamente para esta demostración de laboratorio, no para proteger información real.

```
#include <uECC.h>

// Solo para esta demostracion.
// Este generador NO es seguro para claves reales.
int generarAleatorios(uint8_t* destino, unsigned cantidad) {
    for (unsigned i = 0; i < cantidad; i++) {
        destino[i] = (uint8_t)random(256);
    }
    return 1;
}

void setup() {
    Serial.begin(9600);
    delay(1000);

    randomSeed(12345); // Prueba reproducible
    uECC_set_rng(generarAleatorios);

    // Misma curva que utiliza la filmina.
    const struct uECC_Curve_t* curva = uECC_secp160r1();

    uint8_t clavePrivada[21];
    uint8_t clavePublica[40];
    uint8_t firma[40];

    // SHA-256 del texto "abc", precalculado.
    uint8_t hash[32] = {
        0xba, 0x78, 0x16, 0xbf, 0x8f, 0x01, 0xcf, 0xea,
        0x41, 0x41, 0x40, 0xde, 0x5d, 0xae, 0x22, 0x23,
        0xb0, 0x03, 0x61, 0xa3, 0x96, 0x17, 0x7a, 0x9c,
        0xb4, 0x10, 0xff, 0x61, 0xf2, 0x00, 0x15, 0xad
    };

    Serial.println(F("Generando claves..."));

    if (!uECC_make_key(clavePublica, clavePrivada, curva)) {
        Serial.println(F("Error al generar las claves."));
        return;
    }

    Serial.println(F("Firmando el hash de abc..."));

    if (!uECC_sign(clavePrivada, hash, sizeof(hash),
        firma, curva)) {
        Serial.println(F("Error al generar la firma."));
    }
}
```

```
        return;
    }

    Serial.println(F("Firma generada."));
    Serial.println(F("Verificando con la clave publica..."));

    if (uECC_verify(clavePublica, hash, sizeof(hash),
                    firma, curva)) {
        Serial.println(F("Firma VALIDA."));
    } else {
        Serial.println(F("Firma INVALIDA."));
    }

    // Cambiar un byte para comprobar la integridad.
    hash[0] ^= 1;

    Serial.println(F("Verificando con el hash modificado..."));

    if (uECC_verify(clavePublica, hash, sizeof(hash),
                    firma, curva)) {
        Serial.println(F("Firma VALIDA."));
    } else {
        Serial.println(F("Firma INVALIDA: los datos cambiaron."));
    }
}

void loop() {
    // Se ejecuta una sola vez por reinicio.
}
```

Referencias

- [1] jgamblin. *Mirai-Source-Code*. GitHub. <https://github.com/jgamblin/Mirai-Source-Code>.
- [2] INCIBE. *Introducción a los sistemas embebidos*. <https://www.incibe.es/incibe-cert/blog/introduccion-los-sistemas-embebidos>.
- [3] *Seguridad en sistemas embebidos*. ResearchGate. https://www.researchgate.net/publication/327163345_Seguridad_en_sistemas_embebidos.
- [4] tzikis. *ArduinoMD5*. GitHub. <https://github.com/tzikis/ArduinoMD5>.
- [5] DavyLandman. *AESLib*. GitHub. <https://github.com/DavyLandman/AESLib>.
- [6] kmackay. *micro-ecc*. GitHub. <https://github.com/kmackay/micro-ecc>.